Examen parcial de teoría de SO

Instrucciones para el examen:

- **Justifica todas tus respuestas del examen.** Las respuestas no justificadas se considerarán erróneas.
- Podéis consultar el man instalado en el PC **pero ningún otro material.**
- **Utiliza el fichero template** que os damos junto con este enunciado para responder a las preguntas del examen y **entrégalo a través del racó** (solo un fichero).
- Utiliza un editor **en modo texto**.
- **Utiliza solo el espacio** que hay **reservado** para cada respuesta.
- Si no puedes responder una pregunta deja en blanco el espacio reservado para esa pregunta (no borres ninguna pregunta).

Pregunta Procesos (*5 puntos*)

Tenemos un ordenador que tiene una única unidad de ejecución y un Sistema Operativo Linux como el explicado en clase. Tenemos el siguiente código fuente correspondiente al programa "prog":

```
1.  void main(int argc, char **argv){
2.       char buf[512];
3.       printf("El proceso %d tiene el parametro de entrada %s\n", getpid(), argv[1]);
4.       sprintf(buf, "%d", atoi(argv[1])-1);
5.       if (atoi(buf) > 0) {
6.            if (fork() > 0)
7.                 execlp(argv[0], argv[0], buf, NULL);
8.       }
9.       func();        /* función que ejecuta código de forma activa durante 1 segundo */
10.      printf("El proceso %d finaliza con el valor %s\n", getpid(), argv[1]);
11.      exit(atoi(argv[1]);
12. }
```

figura 1 Código de prog

Contesta a las siguientes preguntas, argumentando brevemente tus respuestas, asumiendo que ejecutamos este código (donde todas las llamadas a sistema se ejecutan correctamente) con la siguiente línea de comandos: **./prog 4**

1) ***(0,5 puntos)*** Indica qué tipos de eventos han provocado ejecutar código del kernel durante la ejecución del programa, asumiendo que no ha existido ninguna interacción humana durante todo este tiempo.

2) ***(0,5 puntos)*** Dibuja la jerarquía de procesos que crea esta ejecución asignando un PID a cada proceso siguiendo la metodología indicada en las transparencias y en clase. Además, explica brevemente los pasos que han ejecutado los procesos para crear la jerarquía representada. Por último, explica brevemente si tenemos un esquema de creación y ejecución de procesos concurrente o secuencial.

3) ***(0,75 puntos)*** Indica los mensajes que se muestran por pantalla, explicando brevemente los pasos que han ejecutado los procesos para mostrarlos. Además, explica si podemos garantizar que los mensajes siempre aparecerán en el mismo orden o no.

4) ***(0,75 puntos)*** Queremos añadir, entre las líneas de código 9 y 10, la llamada a sistema correspondiente que permita que un proceso espere la muerte de su proceso hijo (no hace falta obtener el valor de finalización), pero ha de tener un comportamiento no bloqueante. Indica la línea de código que añadirías y explica brevemente: (1) qué procesos libera cada proceso que ejecuta esta línea; y (2) de qué manera consideras que afectaría al orden de los mensajes mostrados por pantalla comparado con la respuesta del apartado anterior.

A partir del código original de la Figura 1, hacemos un cambio en la línea 6 que pasará a ser "if (fork() == 0)". Contesta, asumiendo haber aplicado este cambio, a las siguientes preguntas:

5) ***(0,5 puntos)*** Dibuja la jerarquía de procesos que crea esta ejecución asignando un PID a cada proceso siguiendo la metodología indicada en les transparencias y en clase. Además, explica brevemente los pasos que han ejecutado los procesos para crear la jerarquía representada. Por último, explica brevemente si tenemos un esquema de creación y ejecución de procesos concurrente o secuencial.

6) ***(0,75 puntos)*** Indica los mensajes que se muestran por pantalla, explicando brevemente los pasos que han ejecutado los procesos para mostrarlos. Además, explica si podemos garantizar que los mensajes siempre aparecerán en el mismo orden o no.

7) ***(0,75 puntos)*** Asumiendo que se ha añadido correctamente la línea de código con la llamada a sistema del apartado "4", explica brevemente en este nuevo caso qué procesos liberan a cada proceso ejecutando esta línea y de qué manera consideras que afectaría al orden de los mensajes mostrados por pantalla comparado con la respuesta del apartado anterior.

8) ***(0,5 puntos)*** ¿Cómo afectaría en el tiempo de ejecución completo del programa si tuviéramos un ordenador con más unidades de ejecución? Además, ¿afectaría a alguna de las respuestas anteriores de este ejercicio? Razona brevemente tu respuesta.

Pregunta Signals (*5 puntos*)

Queremos hacer un programa que cree N procesos que ejecutaran los programas que nos pasan por línea de comandos y controlar el tiempo de ejecución de los procesos hijos. Tenemos el código mostrado en la figura 2 (está incompleto).

```c
void Usage()
{
    char buffer[128];
    sprintf(buffer,"Usage: ejecuta duracion prog1 [prog2... progn]\n");
    write(2,buffer,strlen(buffer));
    exit(1);
}

void f_handler(int s)
{
    if (s == SIGALRM){
    }
    if (s == SIGCHLD){
    }
}

int main(int argc,char *argv[])
{
    if (argc < 3) Usage();
    struct sigaction sa;
    sa.sa_handler = f_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGALRM, &sa, NULL);
    sigaction(SIGCHLD, &sa, NULL);

    for (uint i = 2; i < argc; i++){
        int ret = fork();
        alarm(atoi(argv[1]));
        if (ret == 0){
            execlp(argv[i], argv[i], NULL);
        }
    }
}
```

figura 2 Código del ejercicio de signals

Queremos controlar la ejecución, duración y finalización de estos programas mediante signals. Contesta las siguientes preguntas.

1) ***(1 puntos)*** Si queremos usar los signals SIGALRM, SIGUSR1 y SIGCHLD para controlar la duración, ejecución y finalización de los procesos: Indica qué llamada(s) a sistema deberíamos añadir y en qué línea(s) de código para asegurar que:
   - SIGALRM está desbloqueado en el padre y bloqueado en los hijos
   - SIGUSR1 está bloqueado en el padre y desbloqueado en los hijos
   - SIGCHLD está desbloqueado en el padre y bloqueado en los hijos
2) ***(0,5 puntos)*** Si el proceso padre recibiera un SIGALRM o un SIGCHLD, ¿Qué signal(s) estarían bloqueados/desbloqueados durante la ejecución de la función f_handler()?
3) ***(1,25 puntos)*** Indica cómo sería el contenido de la función f_handler para implementar los requisitos siguientes y qué modificaciones habría que hacer al programa principal para poder implementar la gestión de SIGALRM y SIGCHLD:

- ***(0,5 puntos)*** Si estamos gestionando SIGALRM, queremos enviar un SIGKILL a todos los procesos hijos (no hace falta comprobar si están vivos).
- ***(0,75 puntos)*** Si estamos gestionando el SIGCHLD, queremos imprimir el tiempo que queda hasta el timeout, fin de alarma, pero no cancelar la alarma y, además, queremos recoger el estado de finalización del/los procesos que hayan acabado e imprimir su PID

4) ***(1,25 puntos)*** Queremos limitar el tiempo de ejecución de cada uno de los procesos hijos al número de segundos recibido en atoi(argv[1]). Se nos ocurren estas tres opciones:
   a. ***(0,75 puntos)***¿Alguna de ellas garantiza que el proceso padre reciba el SIGALRM cuando cada uno de los procesos hijos lleve en ejecución los atoi(argv[1]) segundos ?
   b. ***(0,5 puntos)*** Comenta para cada opción si padre e hijos tendrán o no programada la alarma

opción 1

```
for (uint i = 2; i < argc; i++){
  alarm(atoi(argv[1]));
  ret = fork();
```

opción 2

```
alarm(atoi(argv[1]));
for (uint i = 2; i < argc; i++){
```

opción 3

```
for (uint i = 2; i < argc; i++){
 int ret = fork();
 alarm(atoi(argv[1]));
```

5) ***(1 puntos)*** Considera el código con las modificaciones que hayas propuesto en la pregunta 3, haz una propuesta para evitar que el proceso inicial termine antes que sus hijos. Propón una solución lo más eficiente posible en cuanto a consumo de CPU. Indica qué código añadirías y en qué líneas. Restricción: La gestión de la finalización de procesos solo se puede hacer en la atención del SIGCHLD